Estado atual do InStok - Acompanhamento

Análise do desenvolvimento, da arquitetura e das próximas entregas

Identificação	Estado atual
Data de referência	09 de novembro de 2026
Fase	Desenvolvimento ativo do MVP acadêmico
Tecnologias	Python 3, Flask, Jinja2, HTML, CSS, JavaScript e SQLite
Resumo	Autenticação e estrutura base estão implementadas. Os fluxos operacionais de estoque continuam pendentes.

Objetivo e critério de leitura

Este documento resume o estado presente do repositório em relação ao escopo do Modelo de Documentação do Projeto Integrado. Os requisitos do projeto descrevem o resultado esperado; a existência de uma rota, tela ou tabela não comprova que o caso de uso esteja funcional. O banco local não estava disponível nesta cópia, portanto os dados de uma instalação não foram auditados.

Status implementado no código

- Aplicação Flask com factory create_app, blueprints por módulo, templates Jinja2 e configuração por variáveis de ambiente.
- Login por usuário e senha com hash via Werkzeug; sessões; logout via POST; revalidação de usuário ativo e perfil em cada requisição autenticada.
- Proteção CSRF para métodos que alteram estado. As páginas internas exigem autenticação e a área de usuários exige perfil ADMINISTRADOR.
- Conexão SQLite centralizada, chaves estrangeiras habilitadas por conexão e isolamento do banco real durante testes.
- Login e navegação integrados ao frontend visual da equipe, com alternância de tema, visibilidade de senha, página ativa e menu móvel.

Estrutura atual do banco

database/schema.sql define usuarios, categorias, produtos, setores, estoques e movimentacoes, com chaves estrangeiras, restrições de unicidade, CHECKs e índices. O saldo fica em estoques por combinação de produto e setor; produtos armazenam estoque_minimo, não a quantidade atual. database/seed.sql permanece como esqueleto comentado e não póvoa dados.

O README registra 30 testes aprovados em banco temporário em 30/10/2026, com cobertura de autenticação, configuração, banco e scripts. Testes antigos dos módulos de estoque são placeholders e não constituem cobertura funcional.

Arquitetura e componentes atuais

A estrutura segue Route → Service → Repository → Database, conforme o padrão previsto para o MVP. A arquitetura está montada, mas a implementação funcional ainda se concentra em autenticação e operações básicas de usuários.

Camada	Estado no repositório	Situação
Routes	Blueprints para autenticação, dashboard, usuários, produtos, categorias, setores, estoque, movimentações, alertas e relatórios.	Autenticação funciona; várias rotas apenas apresentam páginas.
Services	Auth implementa validação do login. Usuario_service contém validação e cadastro parcial. Os serviços de estoque, movimentação, alerta e cadastros restantes são placeholders.	Regras do estoque ainda não estão codificadas.
Repositories	Usuario_repository contém SQL real. Os repositories de produtos, categorias, setores, estoques e movimentações têm funções TODO/pass.	Persistência operacional pendente.
Database	SQLite via sqlite3 em app/database/connection.py; schema em database/schema.sql.	Não há migrações versionadas.
Apresentação	Templates Jinja2, CSS e JavaScript servidos pelo Flask.	Protótipo visual foi adaptado; não há SPA independente.

Padrão previsto para estoque e rastreabilidade

- O saldo é mantido por produto e setor em estoques. A combinação produto_id + setor_id é única, e a quantidade não pode ser negativa.
- Cada movimentação registra produto, usuário responsável, tipo, quantidade, origem, destino, observação e data/hora.
- Entrada deve creditar um setor de destino. Saída deve debitar uma origem após validar saldo suficiente. Transferência deve debitar a origem e creditar destinos distintos.
- A gravação do movimento e a alteração do saldo devem ocorrer na mesma transação, com rollback integral em caso de falha. Essa coordenação ainda não foi implementada.
- O alerta usa a soma do produto em todos os setores e o estoque mínimo global: total zero significa sem estoque; total positivo até o mínimo significa estoque baixo; acima do mínimo significa normal.

O schema protege algumas invariantes locais, mas não garante sozinho coerência entre tipo e setores nem atomicidade entre movimentação e saldo. Essas regras precisam ser aplicadas pelo service com transação coordenada.

Escopo previsto que ainda falta implementar

Continuam pendentes os CRUDs de categorias, produtos e setores; inventário; entradas, saídas e transferências; histórico pesquisável; alertas; dashboard com dados reais; relatórios e permissões completas para cada perfil.

Backlog técnico e próximos passos

A sequência abaixo respeita as dependências entre cadastros, estoque e consultas. Um item deve ser marcado como concluído somente depois de implementar o fluxo principal e validar seus erros relevantes.

Ordem	Entrega	Critério de conclusão
1	Preparação do banco	Confirmar o banco de desenvolvimento. Criar migrações versionadas antes de alterar estruturas existentes; IF NOT EXISTS não atualiza tabelas.
2	Categorias e setores	Listar, cadastrar, editar e inativar logicamente; validar campos, normalização, duplicidade e vínculos históricos.
3	Produtos	Validar código único, categoria ativa e estoque mínimo inteiro não negativo; preservar histórico ao inativar.
4	Inventário	Consultar saldo por setor e total do produto; criar vínculos produto-setor sem duplicidade; não duplicar saldo em produtos.
5	Movimentações	Validar quantidade positiva, entidades ativas, usuário da sessão, origem/destino e saldo. Atualizar saldo e histórico na mesma transação.
6	Histórico	Listar e filtrar por período, tipo, produto, setor e usuário. Preservar eventos passados.
7	Alertas e dashboard	Calcular estados com saldos reais agregados, incluindo produtos sem posição de estoque.
8	Relatórios e permissões	Gerar relatórios básicos de inventário e movimentos. Definir permissões de administrador e operador por caso de uso.
9	Qualidade e operação	Substituir placeholders; testar regras, permissões, constraints e rollback em banco temporário. Documentar backup e execução.

Validações transversais

- Validar no servidor campos obrigatórios após trim, limites e tipos; apresentar mensagens compreensíveis para duplicidade e referências inválidas.
- Impedir novas movimentações com produto, setor ou usuário inativo.
- Obter usuário responsável e nível de acesso da sessão e do banco, nunca de campos confiados do formulário.
- Cobrir quantidade inválida, saldo insuficiente, setor de origem igual ao destino, falha intermediária com rollback e submissões concorrentes.

Decisões de design e pontos de atenção

Diretrizes para as próximas etapas

- Manter Flask, Jinja2, HTML/CSS/JavaScript e SQLite, com SQL explícito em repositories, conforme a arquitetura do MVP.
- Manter a dependência Route → Service → Repository → Database. Services concentram regras e limites de transação; repositories concentram SQL.
- Manter uma única fonte de saldo em estoques. Toda alteração quantitativa deve gerar uma movimentação rastreável.
- Usar inativação lógica para registros referenciados. Definir política de ajustes e estornos antes de permitir correções no histórico.
- Não usar CREATE TABLE IF NOT EXISTS como mecanismo de atualização; adotar migrações quando houver evolução do schema.
- Manter testes isolados do banco local. Antes da exposição pública, revisar limitação de tentativas de login e configuração de hospedagem; usar chave forte fora do Git e cookie Secure sob HTTPS.

Pontos de atenção

Tema	Ação recomendada
Permissões	O código restringe o módulo de usuários a administradores; definir a matriz completa de permissões para operadores e demais operações.
Histórico	Evitar exclusão física de cadastros relacionados e de movimentos. Usar inativação lógica e correção auditável.
Tipos de quantidade	O schema usa INTEGER. Confirmar que o escopo trabalha apenas com unidades inteiras antes de implementar os formulários.
Banco local	O arquivo de banco não foi encontrado nesta cópia. Confirmar o alvo antes de inicializar, migrar ou corrigir dados.

Autoria e apoio

Este documento foi elaborado com auxílio de LLM e estudos de Engenharia de Prompt, a partir da análise do código e dos materiais do projeto. Os documentos de referência e os diagramas são de autoria dos responsáveis pelo projeto, desenvolvidos em grupo.

Atualizar esta referência após cada entrega, distinguindo requisitos, funcionalidades implementadas e testes efetivamente executados.